04강

프로그래밍 언어론

구문론과 의미론

컴퓨터과학과 김진욱교수

학습목차



1. 구문론과 의미론
2. 구문의 표현
3. 의미의 표현

01

구문론과 의미론

1. 구문론과 의미론

언어의 형식적 정의

- 구문론과 의미론을 통해 언어를 엄밀하게 정의

구문론(syntax)

- 문장의 올바른 형태를 정의

의미론(semantics)

- 올바른 형태의 문장이 나타내는 의미를 정의

1. 구문론과 의미론

언어의 형식적 정의

- 구문론과 의미론을 통해 언어를 엄밀하게 정의

한국어: 나는 너를 사랑한다.

- 구문: 주어 + 목적어 + 서술어
- 의미: 화자가 청자를 몹시 아끼고 귀중히 여긴다.

영어: I love you.

- 구문: 주어 + 동사 + 목적어
- 의미: 화자가 청자를 몹시 아끼고 귀중히 여긴다.

1. 구문론과 의미론

프로그래밍 언어의 형식적 정의

- 프로그래밍 언어의 명확한 구문과 의미를 정의
- 이를 통해 명확한 사용체계를 제공

BASIC: PRINT "GCD is"; A

- 구문: PRINT "출력할 내용"; 변수
- 의미: 출력할 내용과 변수의 값을 순차적으로 출력하라.

C: printf("GCD is %d", a);

- 구문: printf("출력할 내용", 변수);
- 의미: 출력할 내용의 %d 자리에 변수의 값을 대신 넣어 내용을 출력하라.

1. 구문론과 의미론

프로그래밍 언어의 형식적 정의

- 프로그래밍 언어의 명확한 구문과 의미를 정의
- 이를 통해 명확한 사용체계를 제공

형식적 정의의 필요성

- 컴퓨터: 프로그램 해석의 모호함 제거
- 작성자: 프로그램의 동작 예측 가능

1. 구문론과 의미론

프로그래밍 언어의 형식적 정의

프로그램의 구조

- 문자: 영어 알파벳, 아라비아 숫자, 특수 기호 등
- 어휘(토큰): 문자의 나열. 최소한의 의미를 갖는 단어
- 구문: 프로그램을 작성하는 규칙. 토큰을 모아 프로그램을 구성

```
int x12;
x12 = 1 + 5 * 2;
if x12>10 then ...
```

1. 구문론과 의미론

프로그래밍 언어의 형식적 정의

프로그램의 의미

- 프로그램을 통해 발생하는 현상
  - 정수를 저장할 변수 x12를 만든 다음
  - 수식을 계산하여 11을 변수 x12에 대입하고
  - 변수 x12의 값이 10보다 크므로 .. 부분을 수행

```
int x12;
x12 = 1 + 5 * 2;
if x12>10 then ...
```

1. 구문론과 의미론

프로그래밍 언어의 형식적 정의

구문론

- 프로그램의 표면적인 구조를 정의
- 프로그램 작성 시 어떤 형태로 작성해야 하는지를 기술

의미론

- 프로그램의 내용적인 효과를 정의
- 프로그램 실행 시 어떤 일이 일어나는지 그 의미를 기술

02

구문의 표현

2. 구문의 표현

구문론

- 프로그램의 표면적인 구조를 정의
- 정의된 구문을 통해 모든 정상적인 프로그램을 도출할 수 있음
- 작성된 프로그램이 정의된 구문에 맞는 프로그램인지 확인 가능

구문의 표현

- 명확한 표현을 위해 문법을 활용
- 일반적으로 프로그래밍 언어에서는 문맥 자유 문법을 이용

2. 구문의 표현

문맥 자유 문법(CFG: Context-Free Grammar)

- 구성: 비단말 기호 집합, 단말 기호 집합, 생성 규칙 집합, 시작 비단말 기호
- 생성 규칙은 하나의 비단말 기호를
  단말 기호와 비단말 기호의 조합으로 정의

용어

- 비단말 기호: 정의될 대상
- 단말 기호: 언어에서 직접 사용되는 표현
- 생성 규칙: 어떤 비단말 기호를 어떻게 바꾸어 쓸 수 있는지 규정
- 시작 비단말 기호: 언어에 속한 모든 형태를 만들 수 있는 비단말 기호

2. 구문의 표현

문맥 자유 문법(CFG: Context-Free Grammar)

- 구성: 비단말 기호 집합, 단말 기호 집합, 생성 규칙 집합, 시작 비단말 기호
- 생성 규칙은 하나의 비단말 기호를
  단말 기호와 비단말 기호의 조합으로 정의

문맥 자유 문법 G = (N, T, P, E)

P

```
E → E + E
E → E * E
E → D
D → 0
D → 1
```

- 비단말 기호 집합 N = {E, D}
- 단말 기호 집합 T = {+, *, 0, 1}
- 생성 규칙 집합 P
- 시작 비단말 기호 E

2. 구문의 표현

- 문맥 자유 문법의 다양한 표현 방법
  - BNF
  - EBNF
  - 구문 도표

세 가지 표현법은 서로 변환이 가능

- BNF ↔ EBNF
- EBNF ↔ 구문 도표
- 구문 도표 ↔ BNF

2. 구문의 표현

BNF(Backus-Naur Form)

- Algol의 구문을 정의하기 위해 사용된 표현법

세 가지 메타기호

::=	정의
|	택일
< >	비단말 기호

그외 메타기호

' '	단말 기호를 묶음
;	규칙 묶음을 구별하기 위해 각 규칙의 마지막에 사용

2. 구문의 표현

BNF(Backus-Naur Form)

- 비단말 기호: 메타 기호 < >로 묶인 기호
- 단말 기호: 비단말 기호 및 메타 기호가 아닌 기호
- 규칙: 메타 기호 ::=를 기준으로 왼쪽 부분을 오른쪽 부분으로 정의

```
<if문> ::= if <논리식> then <문장> else <문장>
         | if <논리식> then <문장>
         ;
```

2. 구문의 표현

EBNF(Extended Backus-Naur Form)

- BNF에 추가적인 메타 기호를 사용하여 규칙을 보다 간결하게 표현

추가된 메타기호

[ ]	생략 가능
{ }	0번 이상 반복
( )	묶음. 메타 기호 |과 함께 쓰여 한정된 범위의 택일

그외 메타기호

' '	단말 기호를 묶음. 메타 기호를 단말 기호로 사용
;	규칙 묶음을 구별하기 위해 각 규칙의 마지막에 사용

2. 구문의 표현

EBNF(Extended Backus-Naur Form)

- [ ] 생략가능

```
<if문> ::= if <논리식> then <문장> [ else <문장> ]
         ;
```

- BNF 표현

```
<if문> ::= if <논리식> then <문장> else <문장>
         | if <논리식> then <문장>
         ;
```

2. 구문의 표현

EBNF(Extended Backus-Naur Form)

- { } 0번 이상 반복

```
<unsigned integer> ::= <digit> { <digit> }
                     ;
```

- BNF 표현

```
<unsigned integer> ::= <digit>
                     | <unsigned integer><digit>
                     ;
```

2. 구문의 표현

EBNF(Extended Backus-Naur Form)

- ( ) 묶음. 메타 기호 |과 함께 쓰여 한정된 범위의 택일

```
<수식> ::= <수식> ( + | - | * | / ) <수식>
         ;
```

- BNF 표현

```
<수식> ::= <수식> + <수식> | <수식> - <수식>
         | <수식> * <수식> | <수식> / <수식>
         ;
```

2. 구문의 표현

- EBNF(Extended Backus-Naur Form)
  - ‘ ’ 단말 기호를 묶음. 메타 기호를 단말 기호로 사용

```
<BNF 규칙> ::= <왼쪽 부분> ‘::=‘ <오른쪽 부분> ‘;‘
                ;
```

2. 구문의 표현

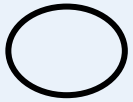
구문 도표(syntax diagram)

- 초기 Pascal의 사용자 설명서에 사용된 표현법

순서도와 유사하게 그림으로 구문을 표현



비단말 기호



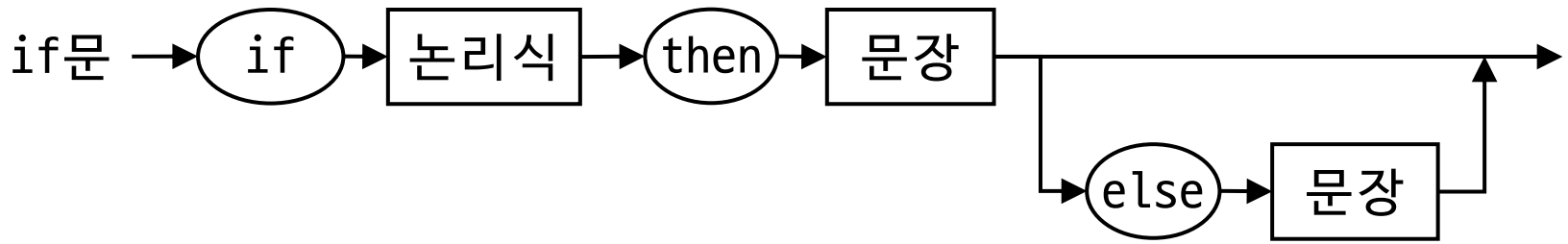
단말 기호



비단말 기호 및 단말 기호들을 연결. 규칙

2. 구문의 표현

구문 도표(syntax diagram)

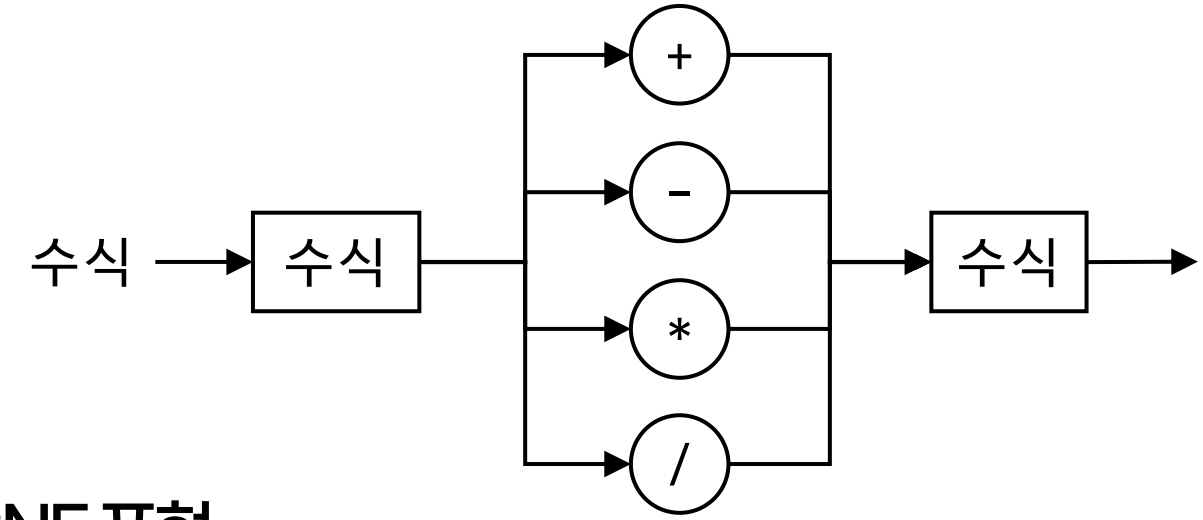


- EBNF 표현

```
<if문> ::= if <논리식> then <문장> [ else <문장> ]
          ;
```

2. 구문의 표현

구문 도표(syntax diagram)

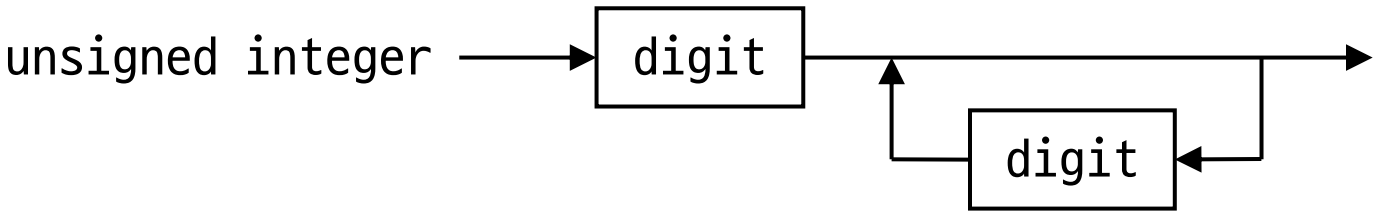


- EBNF 표현

```
<수식> ::= <수식> ( + | - | * | / ) <수식>
         ;
```

2. 구문의 표현

구문 도표(syntax diagram)



- EBNF 표현

```
<unsigned integer> ::= <digit> { <digit> }
                       ;
```

03

의미의 표현

3. 의미의 표현

의미론

- 프로그램의 내용적인 효과를 정의
- 프로그램 실행 시 어떤 일이 일어나는지 그 의미를 기술
- 구문으로 표현하기 어려운 제약사항을 기술하기도 함

의미의 표현

- 일반적으로 자연어 문장으로 표현하나 명확성이 부족
- 의미를 엄밀히 표현하기 위해 다양한 기법이 개발됨 (형식 의미론)

3. 의미의 표현

두 가지 의미론

정적 의미론

- 프로그램을 수행하기 전 의미가 맞는지 파악하는 방법
- 주로 타입 검사 수행에 활용
- 대표적인 표현 방법: 속성 문법

동적 의미론

- 프로그램 수행 시 나타나게 될 의미를 표현하는 방법
- 대표적인 표현 방법: 기능적 의미론, 표기적 의미론, 공리적 의미론 등

3. 의미의 표현

속성 문법

- 정적 의미론의 표현 방법
- 각 비단말 기호마다 타입 속성이 있다고 가정하여 이에 대한 규칙을 정의

구문 규칙	속성 계산 규칙
<D> ::= <T> <id> <L> ';' ;	id.t = T.t ∧ L.t = T.t
<T> ::= int	T.t = 정수
<T> ::= char	T.t = 문자
<L> ::= <id> $<L_1>$	id.t = L.t ∧ L_1.t = L.t
<L> ::= ε	

3. 의미의 표현

기능적 의미론

- 동적 의미론의 표현 방법 중 한 가지
- 추상기계의 상태를 바꾸는 것으로 수행 의미를 표현
  - 프로그램 수행은 결국 컴퓨터의 상태를 바꾸는 것

- 상태: <수행할 명령어, 메모리 상태>

```
 < z=x; x=y; y=z; , [x→5, y→7, z→0] >
⇒ <     x=y; y=z; , [x→5, y→7, z→5] >
⇒ <          y=z; , [x→7, y→7, z→5] >
⇒ <               , [x→7, y→5, z→5] >
```

3. 의미의 표현

표기적 의미론

- 동적 의미론의 표현 방법 중 한 가지
- 각 구문 요소를 수학적 표기에 대응시켜 수행 의미를 표현
- 의미함수: 대응시키는 함수

구문 규칙	의미함수 Bin
<B> ::= 0 | 1 | <B> 0 | <B> 1 ;	Bin[[0]] = 0 Bin[[1]] = 1 Bin[[B 0]] = 2 * Bin[[B]] Bin[[B 1]] = 2 * Bin[[B]] + 1

3. 의미의 표현

공리적 의미론

- 동적 의미론의 표현 방법 중 한 가지
- 프로그램의 효과로 수행 의미를 표현
  - 프로그램의 효과: 프로그램 S가 실행됨으로써 사전조건 P를 사후조건 Q로 변화시킴. {P} S {Q}
- 공리 체계를 이용함으로써 주변 상황에 대응하는 논리식을 정확히 구할 수 있음
- 대입문의 효과 공리: {Q[x→E]} x = E; {Q}

```
{P} a = b * 2; {a < 10}

 P ⇔ (a < 10)[a→b*2] ⇔ b * 2 < 10 ⇔ b < 5

{b < 5} a = b * 2; {a < 10}
```

3. 의미의 표현

❑ 의미론의 한계 및 효과

의미론의 한계

- 프로그래밍 언어 전체에 대한 의미 표현은 너무 복잡함

의미론의 효과

- 프로그램의 구현 및 분석 등에 유용하게 사용됨
- 속성 문법: 인터프리터 및 컴파일러 구현 시
  트리 생성, 타입 검사, 코드 생성 등에 유용
- 수학적 표기: 언어의 특성을 명확하게 정의해야 할 때
- 공리적 의미론: 프로그램의 특정 조건 만족 여부를 확인할 때

정리하기

- 프로그래밍 언어는 문장을 정확하게 이해할 수 있도록 명확한 구문과 그 의미가 정의되어야 함. 이를 프로그래밍 언어의 형식적 정의라 하며 이는 다시 구문론과 의미론으로 나뉨
- 프로그래밍 언어에서 구문은 문맥 자유 문법으로 표현됨. 문맥 자유 문법이란 문맥에 영향을 받지 않는 문법을 의미
- 문맥 자유 문법을 표현하는 방법으로 BNF, EBNF, 구문 도표가 있음
- 프로그래밍 언어에서 의미는 자연어로 표현되는 경우가 많지만, 엄밀한 표현을 위한 형식 의미론도 존재
- 형식 의미론에서 다루는 표현 방법으로는 속성 문법, 기능적 의미론, 표기적 의미론, 공리적 의미론이 있음

05강

다음시간안내 ▶▶▶

구문 분석